

UNIVERSIDAD DE LAS FUERZAS ARMADAS – ESPE

DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN

CARRERA DE INGENIERÍA EN SOFTWARE

ASIGNATURA: DESARROLLO DE VIDEOJUEGOS

UNIDAD 1

Tres en Raya con Inteligencia Artificial

Proyecto 3: Tic-Tac-Toe (UnbeatableAI) – Go (Web) + Angular

DOCENTE: ING. CÉSAR VILLACÍS SILVA, MSC.

ESTUDIANTES:

Diana Guerra
Paul Moreno
Germán Cáceres

GRUPO N.º 3

FECHA: NOVIEMBRE 2026

Índice

1. Proyectos Propuestos del Capítulo 1 **2**

2. Especificación de Requerimientos (IEEE 830) **2**
2.1. Introducción . . . 2
2.1.1. Propósito . . . 2
2.1.2. Alcance . . . 2
2.1.3. Definiciones, acrónimos y abreviaturas . . . 2
2.1.4. Referencias . . . 3
2.2. Descripción general . . . 3
2.2.1. Perspectiva del producto . . . 3
2.2.2. Funciones del producto . . . 3
2.2.3. Características de los usuarios . . . 3
2.2.4. Restricciones . . . 3
2.2.5. Suposiciones y dependencias . . . 3
2.3. Requerimientos específicos . . . 3
2.3.1. Requerimientos funcionales . . . 3
2.3.2. Requerimientos no funcionales . . . 4

3. Modelo de Inteligencia Artificial **5**

4. Diagramas **6**
4.1. Diagrama de casos de uso . . . 6
4.2. Diagrama de secuencia: marcar casilla . . . 7
4.3. Diagrama de clases (backend Go) . . . 8

5. Metodología SCRUM + XP **8**
5.1. Roles . . . 8
5.2. Product Backlog . . . 9
5.3. Sprints . . . 9
5.4. Prácticas XP aplicadas . . . 9
5.5. Diseño . . . 10
5.6. Desarrollo . . . 10
5.7. Pruebas . . . 11

6. Análisis de Resultados: Métricas de Software **11**
6.1. Líneas de código . . . 11
6.2. Otras métricas . . . 12
6.3. Hallazgo sobre la lógica original . . . 12

7. Conclusiones y Recomendaciones **12**
7.1. Conclusiones . . . 12
7.2. Recomendaciones . . . 12

1. Proyectos Propuestos del Capítulo 1

El docente propuso implementar el juego del Tres en Raya con niveles de dificultad en distintas tecnologías. Al Grupo 3 le corresponden los proyectos web en Go y JavaScript:

Tabla 1: Proyecto 3 – Grupo 3

N.º	Variante de IA	Tecnología	Estado
1	UnbeatableAI	Go (Web) + Angular	**Implementado**
2	MediumAI	Go (Web) + Angular	Apartado preparado
3	UnbeatableAI	JavaScript (Web)	–
4	MediumAI	JavaScript (Web)	–
5	UnbeatableAI → MinMax	Go (Web) + Angular	Apartado preparado
6	UnbeatableAI → MinMax	JavaScript (Web)	–

El equipo eligió **Go** para el backend y **Angular** para el frontend. Por especificación, la lógica del juego es *exactamente la misma* que la del proyecto de referencia en C# (`TicTacToeGameWindows2`); solo cambia la tecnología.

2. Especificación de Requerimientos (IEEE 830)

2.1. Introducción

2.1.1. Propósito

Este documento especifica los requerimientos del videojuego educativo *Tres en Raya con IA* en su versión web. Va dirigido al docente de la asignatura y a los integrantes del grupo.

2.1.2. Alcance

El sistema permite que un usuario juegue al Tres en Raya contra un *Jugador Controlado por la Computadora* (JCC) desde un navegador. La JCC decide sus jugadas con el método numérico heurístico de la referencia (pesos usuario = 1, JCC = 3, vacío = 0). El nivel implementado es **UnbeatableAI**; la arquitectura deja preparados los niveles **MediumAI**, **UnbeatableAI → MinMax**, **Principiante** y **Dos jugadores**.

2.1.3. Definiciones, acrónimos y abreviaturas

JCC Jugador Controlado por la Computadora (marca “0”).

Usuario Jugador humano (marca “X”); siempre empieza.

mArrayMoves Vector V_k de 9 posiciones con los pesos 0/1/3.

API REST Interfaz HTTP/JSON entre el frontend y el backend.

SPA *Single Page Application*, aplicación web de una sola página.

2.1.4. Referencias

- IEEE Std 830-1998, *Recommended Practice for Software Requirements Specifications*.
- Villacís, C. et al., *Optimización del Juego del Tres en Raya con Niveles de Dificultad, Utilizando Técnicas Heurísticas de IA*, ESPE.
- Material de clase: *Caso 1*, *Caso 2* y *Caso 3 TTT*.

2.2. Descripción general

2.2.1. Perspectiva del producto

Es una aplicación cliente–servidor. El **backend en Go** contiene *toda* la lógica del juego (port 1:1 de las clases C#) y la expone como API REST. El **frontend en Angular** solo dibuja el tablero, envía los clics y anima los eventos que devuelve el servidor.

2.2.2. Funciones del producto

Seleccionar modo de juego, iniciar una partida nueva, marcar una casilla, recibir la jugada de la JCC, conocer el resultado ("Ganó X", "Ganó 0" o "Empate"), ver el marcador y ver cómo razona la IA (pesos y sumas por línea).

2.2.3. Características de los usuarios

Niños de 7 a 11 años (público objetivo del juego original) y estudiantes. No se requiere conocimiento técnico: basta con hacer clic o usar las teclas 1–9.

2.2.4. Restricciones

- La lógica del juego no puede modificarse respecto al original.
- Backend en Go 1.23 o superior, solo con la librería estándar.
- Frontend en Angular 22 con navegadores modernos (Chrome, Edge, Firefox).

2.2.5. Suposiciones y dependencias

El servidor Go está disponible en el puerto 8080; en desarrollo, Angular redirige `/api` mediante `proxy.conf.json`.

2.3. Requerimientos específicos

2.3.1. Requerimientos funcionales

Tabla 2: Requerimientos funcionales

ID	Descripción	Prioridad
RF01	El sistema mostrará un tablero de 3×3 casillas numeradas 0–8 (TabIndex).	Alta

ID	Descripción	Prioridad
RF02	El usuario podrá seleccionar el modo de juego; los modos no implementados se mostrarán como "en construcción".	Alta
RF03	El usuario siempre inicia con la marca "X".	Alta
RF04	Al marcar una casilla libre, el sistema registrará peso 1 en `mArrayMoves`; una casilla ocupada se ignora.	Alta
RF05	En su primera jugada la JCC tomará el centro (4) si está libre; si no, una esquina aleatoria (0, 2, 6, 8).	Alta
RF06	La JCC aplicará el método numérico: suma 6 ⇒ gana; suma 2 ⇒ bloquea; suma 5 en diagonal ⇒ caso trivial; dos lados adyacentes del usuario ⇒ esquina intermedia; en otro caso, casilla aleatoria.	Alta
RF07	Si la JCC está en el centro y la regla de suma 5 aplica, jugará un lado aleatorio (1, 3, 5, 7).	Alta
RF08	El sistema verificará ganador en el orden original (8 líneas de "0", 8 de "X", empate) y mostrará el mensaje correspondiente.	Alta
RF09	Al terminar la partida el tablero se deshabilitará (equivalente a `grbTicTacToe.Enabled = false`).	Alta
RF10	El usuario podrá iniciar un juego nuevo en cualquier momento.	Media
RF11	El sistema mostrará un marcador de partidas ganadas, empatadas y perdidas por modo.	Media
RF12	El sistema mostrará los pesos de `mArrayMoves` y la suma de cada línea (inspector de IA).	Baja

2.3.2. Requerimientos no funcionales

Tabla 3: Requerimientos no funcionales

ID	Tipo	Descripción
RNF01	Fidelidad	La lógica de la IA será idéntica a `ControlStrongIntelligentMoves.cs`.
RNF02	Rendimiento	El servidor responderá a cada jugada en menos de 50 ms.
RNF03	Usabilidad	Interfaz animada, con sonido opcional y soporte de teclado (1–9 y teclado numérico).
RNF04	Accesibilidad	Respeta `prefers-reduced-motion` y usa etiquetas ARIA en el tablero.
RNF05	Portabilidad	Funciona en navegadores modernos y es adaptable a celular, tablet y escritorio.
RNF06	Mantenibilidad	Cada modo vive en su propio archivo y cumple una interfaz común (`Controller`).
RNF07	Fiabilidad	Ninguna petición puede bloquear el servidor (protección ante bucles infinitos del original).

ID	Tipo	Descripción
RNF08	Concurrencia	Varias partidas simultáneas, aisladas por sesión, con exclusión mutua.
RNF09	Calidad	Pruebas automatizadas con `go test` para los Casos 1, 2 y 3.

3. Modelo de Inteligencia Artificial

El tablero se representa como el vector V_k, con $k = 0, \ldots, 8$, en el arreglo `mArrayMoves`. Cada casilla vale 0 (vacía), 1 (usuario) o 3 (JCC). Para cada una de las 8 líneas se calcula la suma de sus tres casillas:

Tabla 4: Interpretación de la suma por línea

Suma	Composición	Acción de la JCC
6	$3 + 3 + 0$	Completar la línea y **ganar** (Caso 2)
2	$1 + 1 + 0$	**Bloquear** al usuario (Caso 1)
5 (diagonal)	$1 + 3 + 1$	Caso trivial: esquina libre o lado aleatorio (Caso 3)

La regla de bloqueo en Go conserva el mismo orden de evaluación que el C#:

```
// pick devuelve el primer casillero vacío de la línea (a, b, d).
func (c *ControlStrongIntelligentMoves) pick(a, b, d int) int {
       index := 0
       if c.ArrayMoves[a] == 0 { index = a
       } else if c.ArrayMoves[b] == 0 { index = b
       } else if c.ArrayMoves[d] == 0 { index = d }
       c.sum = c.ArrayMoves[a] + c.ArrayMoves[b] + c.ArrayMoves[d]
       return index
}
```

4. Diagramas

4.1. Diagrama de casos de uso

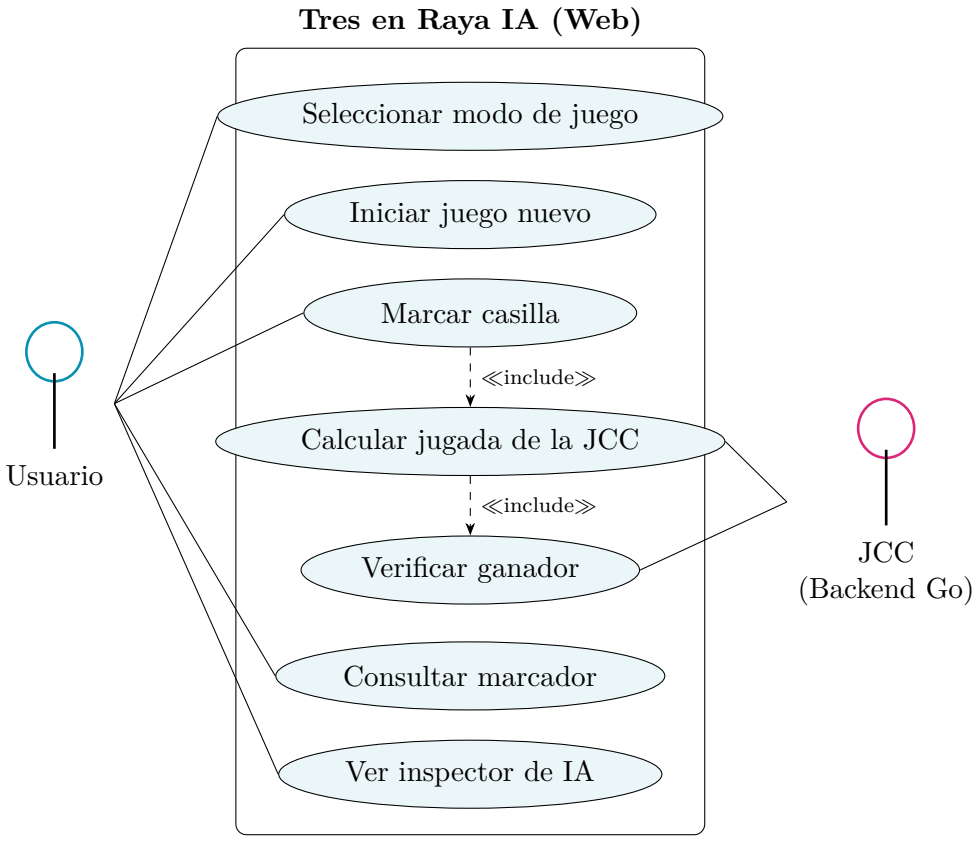


Figura 1: Diagrama de casos de uso

4.2. Diagrama de secuencia: marcar casilla

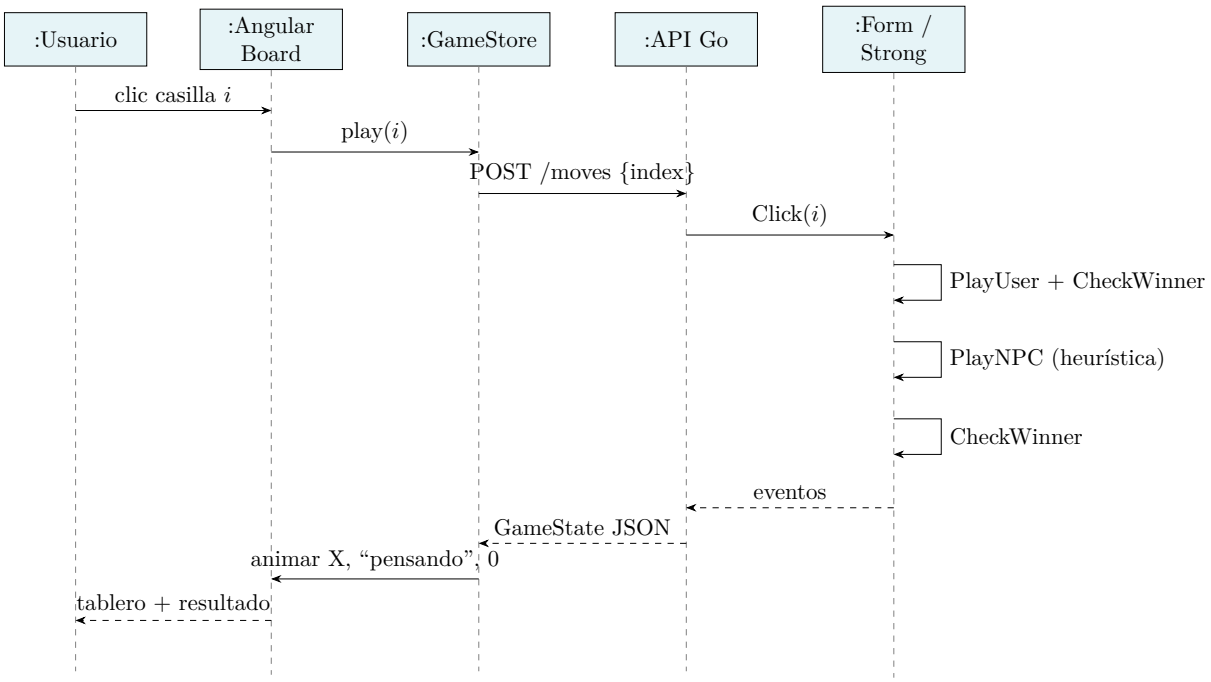


Figura 2: Diagrama de secuencia del caso de uso “Marcar casilla”. Si hay ganador o empate, `Board.Enabled` pasa a `false` y el frontend muestra el resultado con animación.

4.3. Diagrama de clases (backend Go)

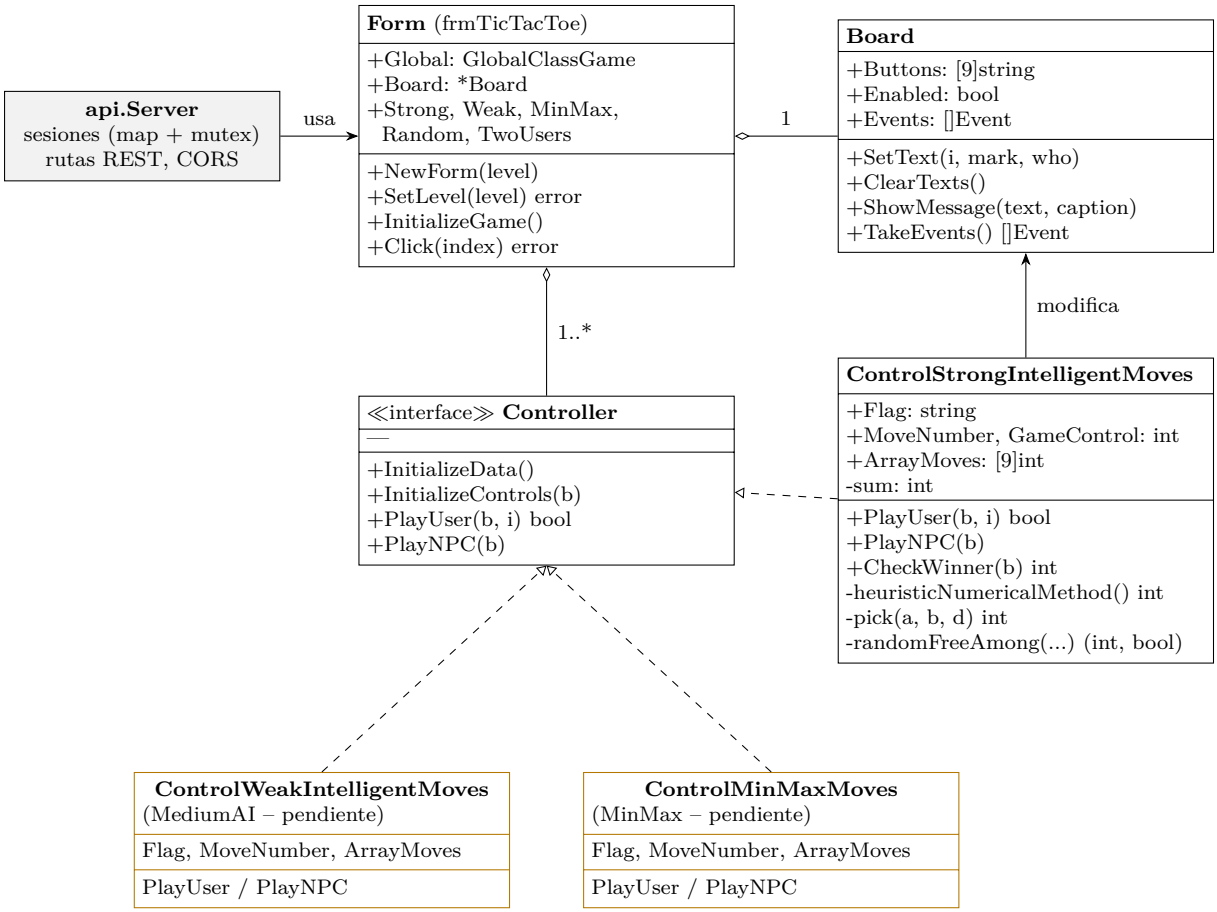


Figura 3: Diagrama de clases del backend

5. Metodología SCRUM + XP

Se combinó **SCRUM** para organizar el trabajo en *sprints* y **XP** para las prácticas de ingeniería.

5.1. Roles

Tabla 5: Roles del equipo

Rol	Responsable
Product Owner	Ing. César Villacís Silva, MSc.
Scrum Master	Germán Cáceres
Desarrollo – UnbeatableAI (Go + Angular)	Germán Cáceres
Desarrollo – MediumAI	Diana Guerra / Paul Moreno
Desarrollo – UnbeatableAI → MinMax	Diana Guerra / Paul Moreno

Pendiente: confirmar con el grupo la asignación final de MediumAI y MinMax.

5.2. Product Backlog

Tabla 6: Historias de usuario

HU	Historia	Pts	Sprint
HU01	Como usuario quiero ver un tablero 3×3 para poder jugar.	3	1
HU02	Como usuario quiero marcar una casilla y que la JCC responda.	5	1
HU03	Como usuario quiero enfrentar a una IA que gane y bloquee (UnbeatableAI).	8	1
HU04	Como usuario quiero saber quién ganó o si hubo empate.	3	1
HU05	Como usuario quiero elegir el modo de juego.	3	2
HU06	Como usuario quiero animaciones y sonidos para que el juego sea atractivo.	5	2
HU07	Como estudiante quiero ver los pesos y sumas que usa la IA.	3	2
HU08	Como usuario quiero un marcador de partidas.	2	2
HU09	Como usuario quiero jugar contra MediumAI.	5	3
HU10	Como usuario quiero jugar contra UnbeatableAI con MinMax.	8	3

5.3. Sprints

Sprint 1 – Núcleo del juego. Port del método numérico a Go (`strong.go`), tablero (`board.go`), despacho de clics (`form.go`), API REST y pruebas de los Casos 1–3.

Sprint 2 – Experiencia web. Frontend Angular 22: tablero 3D con trazos animados, fondo de partículas, sonidos sintetizados, inspector de IA, marcador y pantalla de resultado con confeti.

Sprint 3 – Modos restantes. MediumAI y MinMax sobre la interfaz `Controller` ya preparada.

Pendiente: Sprint 3 – documentar aquí el avance de MediumAI y MinMax.

5.4. Prácticas XP aplicadas

- **Diseño simple:** un archivo por modo y una interfaz común.
- **Pruebas primero:** los ejemplos de los PDF se escribieron como pruebas (`strong_test.go`) antes de conectar la interfaz.
- **Estándar de código:** `gofmt`/`go vet` en Go y Prettier en Angular.
- **Integración continua y entregas pequeñas:** cada incremento se sube al repositorio `github.com/Gpcaceres/videojuegos-grupo3`.
- **Propiedad colectiva:** el `README` explica cómo completar cada modo.

5.5. Diseño

La arquitectura separa responsabilidades en dos capas:

- **Backend (Go):** `internal/game` contiene la lógica (sin dependencias web) e `internal/api` expone las rutas. `Board` reemplaza a los controles de Windows Forms (`ListButtons`, `GroupBox` y `MessageBox`), de modo que el código de los controladores queda casi línea por línea igual al original.
- **Frontend (Angular):** componentes *standalone* con *signals*, sin `zone.js`. `GameStore` reproduce los eventos del backend con pausas para animarlos y no decide ninguna jugada.

Tabla 7: API REST

Método	**Ruta**	**Equivale a (original)**
GET	/api/modes	RadioButtons de nivel
POST	/api/games	Abrir el formulario
POST	/api/games/{id}/moves	`btnTicTacToeN_Click`
POST	/api/games/{id}/new	`btnNew_Click`
PUT	/api/games/{id}/level	`rdbXxx_CheckedChanged`

5.6. Desarrollo

La correspondencia entre el original y el port es directa:

Tabla 8: Correspondencia C# → Go

C# (original)	**Go**
`ListButtons[i].Text`	`Board.Buttons[i]`
`grbTicTacToe.Enabled`	`Board.Enabled`
`MessageBox.Show(...)`	`Board.ShowMessage(...)` → evento
`new Random().Next(0, 9)`	`rand.Intn(9)`
`mSum` (miembro)	`sum` (se conserva entre turnos)
`GlobalClassGame` (estática)	`GlobalClassGame` por sesión

La única diferencia deliberada es la protección de los `do/while` aleatorios: si no queda ninguna casilla del grupo pedido, el original entra en un bucle infinito. En el servidor esto bloquearía la petición, así que se continúa con la jugada heurística (RNF07).

5.7. Pruebas

Tabla 9: Resultados de `go test ./...`

Prueba	Verifica	Resultado
TestFirstMoveTakesCenter	La JCC toma el centro en su primera jugada	PASS
TestFirstMoveCornerWhenCenterTaken	Si el centro está ocupado, toma una esquina (50 repeticiones)	PASS
TestCase1Blocks	Caso 1: bloquea al usuario (suma 2)	PASS
TestCase2Wins	Caso 2: completa la línea y gana (suma 6)	PASS
TestTakesImmediateWin	Prioriza ganar en la diagonal	PASS
TestOriginalLogicExhaustive	Recorre todas las partidas posibles	PASS

Además se verificó la aplicación completa en el navegador (front + back): la JCC responde en el centro, el resultado se muestra al terminar y los modos pendientes aparecen bloqueados.

Pendiente: agregar capturas de pantalla de la aplicación en `Img/` (por ejemplo `captura-juego.png`).

6. Análisis de Resultados: Métricas de Software

6.1. Líneas de código

Se contaron las líneas efectivas de código (sin comentarios ni líneas en blanco):

Tabla 10: Líneas de código por proyecto

Proyecto	Archivos	LOC	Coment.	Blanco
Referencia Java (NetBeans)	8	2 347	291	210
Referencia C# (Windows Forms)	7	1 708	455	195
Backend Go (este proyecto)	11	863	132	114
Frontend Angular (TS + HTML + SCSS)	24	2 179	36	292

Tabla 11: Clase de la IA: original frente al port

Archivo	LOC	Funciones/métodos
ControlStrongIntelligentMoves.java	561	13
ControlStrongIntelligentMoves.cs	462	12
strong.go	**196**	**14**

El port en Go reduce un 58 % las líneas de la clase de IA frente al C#. Las mismas reglas se expresan con un `switch` sin repetir bloques, y una función auxiliar `pick` reemplaza los bloques `if/else if` iguales de cada línea. La verificación de ganador se centraliza en `CheckWinnerBoard`, que en el original estaba copiada en cada controlador.

6.2. Otras métricas

- **Pruebas automatizadas:** 6 de 6 aprobadas; la exploración exhaustiva cubre alrededor de 15 000 partidas en 0,02 s.
- **Tamaño del frontend:** 216 kB sin comprimir (62 kB transferidos) en el *build* de producción.
- **Dependencias del backend:** cero (solo librería estándar de Go).
- **Tiempo de respuesta:** menos de 1 ms por jugada en el servidor local.

6.3. Hallazgo sobre la lógica original

La prueba exhaustiva muestra que la JCC del método numérico **pierde en cerca del 1,2 % de los caminos posibles**. Esto no es un error del port: la lógica original cae en `GenerateRandomNPCMove` cuando ninguna regla aplica, y a veces deja al usuario un *tenedor* (dos amenazas a la vez). Ejemplo: X en 0, JCC en 4, X en 7, JCC en 2 (al azar), X en 6. La JCC solo puede tapar una de las amenazas (0–3–6 y 6–7–8), y el usuario gana.

Como la especificación exige conservar la lógica, el comportamiento se mantiene y se documenta. Es justamente la motivación de la variante **UnbeatableAI → MinMax**, que explora el árbol completo de jugadas y no puede caer en un tenedor.

Pendiente: métricas de MediumAI y MinMax (LOC, pruebas, porcentaje de derrotas) cuando estén implementados.

7. Conclusiones y Recomendaciones

7.1. Conclusiones

- Se migró la IA UnbeatableAI de C# a Go sin alterar su lógica. Las pruebas reproducen los Casos 1, 2 y 3 del material de clase.
- Separar la lógica (backend) de la presentación (frontend) permitió sustituir Windows Forms por una interfaz web animada sin tocar el algoritmo.
- El método numérico basado en pesos 1/3/0 es simple y eficiente, pero no es estrictamente imbatible: su caída al azar deja un 1,2 % de caminos perdedores.

7.2. Recomendaciones

- Implementar la variante MinMax para garantizar una IA realmente imbatible.
- Agregar las reglas de “bloquear tenedor” si en el futuro se permite modificar la heurística.
- Desplegar el backend en un servicio en la nube (por ejemplo Cloud Run) y el frontend como sitio estático.

Referencias

[1] IEEE, *IEEE Recommended Practice for Software Requirements Specifications*, IEEE Std 830-1998.

[2] C. Villacís, W. Fuertes, A. Bustamante et al., *Optimización del Juego del Tres en Raya con Niveles de Dificultad, Utilizando Técnicas Heurísticas de Inteligencia Artificial*, Universidad de las Fuerzas Armadas – ESPE.

[3] The Go Authors, *The Go Programming Language Documentation*, `https://go.dev/doc/`.

[4] Google, *Angular Documentation*, `https://angular.dev`.

[5] K. Schwaber y J. Sutherland, *La Guía de Scrum*, 2020.

[6] K. Beck, *Extreme Programming Explained: Embrace Change*, Addison-Wesley, 2004.